

EMOTION DETECTION FROM TEXT

A project report submitted in partial fulfillment of the requirements

for the award of credits to

Machine Learning Lab Course of

Bachelor of Technology

In

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING –
ARTIFICIAL INTELLIGENCE & MACHINE LEARNING (CSM)**

By

(Bhuvanagiri Bhavani)

(23BQ1A4219)



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING – ARTIFICIAL
INTELLIGENCE & MACHINE LEARNING (CSM)**

VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY

(Approved by AICTE and permanently affiliated to JNTUK)

Accredited by NBA and NAAC with 'A' Grade

NAMBUR (V), PEDAKAKANI (M), GUNTUR-522 508

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING –
ARTIFICIAL INTELLIGENCE & MACHINE LEARNING (CSM)
VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY: NAMBUR
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA**



CERTIFICATE

This is to certify that the project titled “**Emotion Detection From Text**” is a bonafide record of work done by **Ms. Bhuvanagiri Bhavani** under the guidance of **Mrs. B. Lalitha Rajeswari, Associate Professor** in partial fulfillment of the requirement for the award of credits to Machine Learning Lab of Bachelor of Technology in Computer Science & Engineering – Artificial Intelligence & Machine Learning (CSM), JNTUK during the academic year 2024-25.

B. Lalitha Rajeswari

Course Instructor

Prof. K. Suresh Babu

Head of the Department

DECLARATION

I, **BHUVANAGIRI BHAVANI (23BQ1A4219)**, hereby declare that the Project Report entitled “**EMOTION DETECTION FROM TEXT** ” done by me under the guidance of Mrs.B. Lalitha Rajeswari, **Associate Professor** is submitted in partial fulfillment of the requirements for the award of degree of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE & ENGINEERING – ARTIFICIAL INTELLIGENCE & MACHINE LEARNING (CSM)**.

DATE :

SIGNATURE OF THE CANDIDATE

PLACE : VVIT ,Nambur.

BHUVANAGIRI BHAVANI

ACKNOWLEDGEMENT

We express our sincere thanks wherever it is due

We express our sincere thanks to the Chairman, Vasireddy Venkatadri Institute of Technology, Sri Vasireddy Vidya Sagar for providing us well equipped infrastructure and environment.

We thank Dr. Y. Mallikarjuna Reddy, Principal, Vasireddy Venkatadri Institute of Technology, Nambur, for providing us the resources for carrying out the project.

We express our sincere thanks to Dr. K. Giribabu, Dean of Academics for providing support and stimulating environment for developing the project.

Our sincere thanks to Dr. K. Suresh Babu, Head of the Department, Department of CSM, for his co-operation and guidance which helped us to make our project successful and complete in all aspects.

We also express our sincere thanks and are grateful to our guide Mrs. B. Lalitha Rajeswari, Associate Professor, Department of CSM, for motivating us to make our project successful and fully complete. We are grateful for his precious guidance and suggestions.

We also place our floral gratitude to all other teaching staff and lab technicians for their constant support and advice throughout the project.

BHUVANAGIRI BHAVANI

(23BQ1A4219)

Table of Contents

Abstract	6
Chapter 1:Introduction.....	7
Chapter 2:Software Rquirements.....	8
Chapter 3:Literature Survey.....	10
Chapter 4:Implementation.....	11
4.1 Data Preprocessing.....	11
4.2 Feature Extraction.....	11
4.3 Label Encoding.....	11
4.4 Model Selection.....	12
4.5 Evaluation Metrics	12
Chapter 5:Results.....	15
5.1 Accuracy.....	15
5.2 Classification Report.....	15
5.3 Confusion Matrix.....	16
5.4 Summary.....	18
Chapter 6:Conclusion.....	19
References.....	21
Appendix.....	22

Abstract

Emotion detection from text is an evolving area within natural language processing (NLP) and artificial intelligence (AI), aiming to bridge the gap between human emotional expression and machine understanding. This project presents a comprehensive machine learning-based framework to classify emotions embedded in textual data, showcasing how computers can be trained to recognize and respond to human feelings such as joy, anger, sadness, fear, and surprise. The implementation is tailored to real-world applications including mental health assessment, customer feedback analysis, sentiment tracking on social media, and development of emotionally intelligent chatbots.

The foundation of the project lies in the acquisition and preprocessing of labeled textual data. Preprocessing involves steps such as lowercasing, stop-word removal, stemming/lemmatization, and vectorization using TF-IDF or word embeddings (e.g., Word2Vec or GloVe). The system uses pandas, NumPy, NLTK, spaCy, and scikit-learn libraries for efficient data handling and natural language processing. Feature extraction converts raw sentences into numerical formats suitable for classification.

Supervised learning algorithms such as Logistic Regression, Support Vector Machines (SVM), Random Forest, and Naive Bayes were tested and evaluated for their performance in emotion classification. Each model is assessed using accuracy, precision, recall, F1-score, and confusion matrix to determine the best-performing model. Hyperparameter tuning and cross-validation further enhance model performance and reliability.

To visualize the emotional trends in the dataset and model predictions, libraries such as matplotlib and seaborn are used. Graphs and heatmaps allow users to gain insights into how emotions are distributed and how accurately the models detect them. The system also supports real-time emotion prediction for custom user input through a simple interactive interface.

This project goes beyond basic sentiment analysis, moving into the finer classification of emotional states. Its modular structure enables easy integration with larger applications such as chatbots, feedback systems, or educational tools. Additionally, it lays the groundwork for future improvements, including deep learning approaches like LSTM or BERT, support for multilingual datasets, and context-aware emotion prediction.

Ethical considerations are central to the project, ensuring data privacy, fairness, and responsible AI practices. All datasets used are publicly available and de-identified, and model predictions are interpreted transparently. This work reflects the importance of empathy in AI, where machines are not only intelligent but also emotionally responsive.

In conclusion, the project highlights the power of machine learning in decoding emotional patterns from text, contributing to the development of emotionally intelligent systems that can interact more naturally and compassionately with humans.

Chapter 1 : Introduction

Emotion is a fundamental aspect of human communication and interaction. It represents a wide range of feelings and mental states such as happiness, sadness, anger, fear, surprise, and disgust. These emotions can be expressed through facial expressions, voice intonation, body gestures, and increasingly, through text — especially in today's digital age where people frequently communicate via emails, social media, messages, and online reviews.

In recent years, the analysis and recognition of emotions from textual data have gained significant attention in the field of Natural Language Processing (NLP) and Artificial Intelligence (AI). Emotion detection from text refers to the task of identifying the emotional tone or state conveyed in a given piece of textual information. It plays a crucial role in enhancing human-computer interactions, understanding user behavior, and building emotionally intelligent systems.

With the rise of digital platforms, there is an abundance of user-generated content available online, making it essential to develop automated systems that can understand and process human emotions. Applications of emotion detection range from customer service, mental health monitoring, recommendation systems, to social media monitoring and virtual assistants.

This project aims to implement a machine learning-based system that can **detect emotions from text data**. By leveraging techniques such as data preprocessing, feature extraction, and supervised classification algorithms, the system is capable of analyzing textual inputs and classifying them into predefined emotional categories.

Natural Language Processing techniques are used to clean and prepare the textual data, while classification algorithms such as Naive Bayes, Support Vector Machine (SVM), or Logistic Regression are employed to learn patterns from labeled datasets. The performance of the models is evaluated using standard metrics such as accuracy, precision, recall, and F1-score to determine the effectiveness of the emotion detection system.

This project not only demonstrates the potential of combining NLP with machine learning but also contributes to the growing field of affective computing by enabling machines to understand and respond to human emotions through text.

Chapter 2 : Software Requirements

This chapter outlines the essential software tools and libraries used in the development of the **Emotion Detection from Text** machine learning project. It includes Python-based frameworks for text preprocessing, model training, and evaluation. Libraries such as **NLTK**, **scikit-learn**, **TensorFlow**, and **Transformers** are employed to effectively handle natural language processing and machine learning tasks.

1. Programming Language

Python 3.7 - Python is widely used in ML/NLP tasks due to its extensive library support.

2. Development Environment

Jupyter Notebook - Ideal for interactive development and visualization.
Alternatively can use: VS Code, PyCharm, or Google Colab.

3. Core Python Libraries

numpy - for numerical computations
pandas - for data manipulation and preprocessing
matplotlib,searborn - for visualizing data

4. Natural Language Processing (NLP) Libraries

nlTK (Natural Language Toolkit) – for text preprocessing like tokenization, stopword removal, stemming

spacy – for advanced NLP tasks

textblob – for basic sentiment analysis

re – regular expressions for text cleaning

5. Machine Learning Libraries

scikit-learn – for traditional ML models like Naive Bayes, SVM, Decision Trees, etc.

tensorflow / keras – for deep learning models (e.g., LSTM, CNN)

pytorch – alternative deep learning framework (if preferred)

6. Transformers / Deep NLP (Optional for Advanced Projects)

transformers (by Hugging Face) – for using pre-trained models like BERT, RoBERTa for emotion classification

7. Text Vectorization Tools

scikit-learn's TfidfVectorizer, CountVectorizer

gensim – for word embeddings (Word2Vec)

keras.preprocessing.text – for tokenization and sequence preparation

8. Model Evaluation and Utilities

sklearn.metrics – for precision, recall, F1-score, confusion matrix

joblib or pickle – for saving trained models

9. Others

tqdm – for showing progress bars during model training

argparse – for command-line execution (if converting to a script)

flask or streamlit – for deploying the model via web app

In conclusion, the software requirements detailed in this chapter provide the foundational tools and libraries essential for building an effective Emotion Detection from Text system. With Python as the core programming language and a suite of powerful libraries for natural language processing, machine learning, and deployment, the project is well-equipped to handle all stages of development efficiently. These tools ensure reliability, scalability, and ease of implementation throughout the project lifecycle.

Chapter 3 : Literature Survey

Emotion detection from text is an emerging and significant field in Natural Language Processing (NLP) and Machine Learning (ML). It focuses on identifying and classifying emotional expressions embedded within textual data. One of the notable references for this project is a research paper sourced from IEEE, which utilizes the "SemEval-2018 Affect in Tweets Distant Supervision Corpus (AIT-2018 Dataset)." This dataset contains a large collection of emotion-labeled tweets and is designed specifically to support research in affective computing. Although it includes a wide range of emotional labels, the researchers clustered these into four fundamental emotions: **anger**, **fear**, **joy**, and **sadness** for effective classification.

Given that tweets often contain noisy and informal content, the dataset underwent comprehensive preprocessing. This included filtering out non-English tweets, converting text to lowercase, removing URLs, mentions, retweet tags, and unnecessary numerical data. Additionally, the preprocessing involved separating hashtags, expanding short forms, tokenizing the text, removing punctuation and stopwords, and applying stemming and lemmatization. Part-of-speech tagging was also employed to enhance linguistic understanding of the tokenized words.

To ensure balanced model training, the dataset was equalized in terms of sample size across each emotion category. For emotion-related word filtering, the authors employed **WordNet-Affect**, a subset of the WordNet lexical database, which contains only affective (emotion-related) words. Furthermore, **EmoSenticNet** was used as an additional lexical resource to strengthen emotional word detection. These tools helped in refining the dataset by isolating words with emotional value.

Feature extraction was carried out using **Term Frequency-Inverse Document Frequency (TF-IDF)**, a widely adopted technique in NLP for weighting words based on their importance in a corpus. The dataset was then split into training and testing sets and evaluated using traditional machine learning algorithms such as **Naive Bayes**, **Decision Tree**, and **Support Vector Machine (SVM)**. Among these, **SVM** delivered the best performance with an accuracy of **88.23%**. The models were further evaluated using performance metrics like **precision** and **recall**, calculated individually for each emotion class.

This literature forms a foundational basis for the current project by highlighting effective preprocessing techniques, feature extraction methods, and comparative analysis of machine learning algorithms for emotion detection in text. The techniques and tools referenced in the IEEE study have been adapted and extended in our project to explore the potential of emotion recognition from textual inputs.

Chapter 4 : Implementation

The implementation of the Emotion Detection from Text project follows a systematic machine learning pipeline comprising data preprocessing, feature extraction, model training, and performance evaluation. The overall goal is to classify text-based inputs—such as sentences or tweets—into one of several emotional categories based on their content. The primary steps and algorithms used are described below.

4.1 Data Preprocessing

The raw dataset consists of tweets with corresponding emotion labels. Since tweets are unstructured and noisy, preprocessing is essential to clean and normalize the text data. The preprocessing steps include:

- ✧ Converting all text to lowercase.
- ✧ Replacing emojis with their respective meanings using a predefined emoji dictionary.
- ✧ Removing URLs, user mentions (@username), hashtags, punctuation, numbers, and extra whitespace using regular expressions.
- ✧ Tokenizing the text using `word_tokenize()` from the NLTK library.
- ✧ Removing stopwords using NLTK's stopwords corpus.
- ✧ Lemmatizing each token using `WordNetLemmatizer` to convert words to their base forms.
- ✧ Removing short tokens and non-alphabetic words to retain only meaningful content.

These steps ensure that the text data fed into the model is clean, consistent, and relevant to emotional context.

4.2 Feature Extraction

After preprocessing, the clean textual data is transformed into numerical format suitable for machine learning models. This is done using:

CountVectorizer: A text vectorization technique that converts the collection of text into a matrix of token counts.

The resulting sparse matrix represents the frequency of words in each tweet, providing input features for classification.

4.3 Label Encoding

The emotion labels (such as “happy”, “sadness”, “hate”, “surprise”, etc.) are encoded into numeric format using `LabelEncoder` from `sklearn.preprocessing`. This transformation is required to train classification models that expect numerical target values.

4.4 Model Selection and Training

The following machine learning algorithms were used and evaluated for emotion classification:

- ✧ Logistic Regression (LogisticRegression from sklearn.linear_model)
- ✧ Multinomial Naive Bayes (MultinomialNB from sklearn.naive_bayes)
- ✧ Random Forest Classifier (RandomForestClassifier from sklearn.ensemble)
- ✧ Support Vector Machine (SVM) using SGDClassifier from sklearn.linear_model

The process includes:

Splitting the dataset into training and testing sets using train_test_split() (typically 75% train, 25% test).

Training the above models on the training dataset.

Predicting the emotions for the test dataset using each model.

Comparing the models based on accuracy and evaluation metrics.

4.5 Evaluation Metrics

The performance of each trained model is evaluated using the following metrics:

- ✧ Accuracy Score – the overall correctness of the model.
- ✧ Confusion Matrix – shows actual vs predicted emotion classes.
- ✧ Classification Report – includes precision, recall, and F1-score for each emotion class.

Among all models, the **Support Vector Machine (SVM)** achieved the highest accuracy, with strong precision and recall values across major emotion categories.

SOURCE CODE

```
# Importing necessary libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import nltk
import string
import re
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer, TfidfTransformer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import SGDClassifier, LogisticRegression
```

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import MultinomialNB
from sklearn import tree
from sklearn.metrics import accuracy_score, precision_score, recall_score,
confusion_matrix, f1_score, classification_report, ConfusionMatrixDisplay
import warnings

# Suppress warnings
warnings.filterwarnings("ignore")

# Load dataset
df = pd.read_csv('./Dataset/text_emotions.csv')
print('Dataset size:', df.shape)
print('Columns are:', df.columns)

# Target variable
Y = df['sentiment']
df.info()

# Plotting sentiment count
sns.countplot(x='sentiment', data=df)

# Load emoji mappings
emojis = pd.read_csv('./Dataset/emojis.txt', sep=',', header=None)
emojis_dict = {i: j for i, j in zip(emojis[0], emojis[1])}
pattern = '|'.join(sorted(re.escape(k) for k in emojis_dict))

# Function to replace emojis with text
def replace_emojis(text):
    text=re.sub(pattern,lambdam:emojis_dict.get(m.group(0)),text, flags=re.IGNORECASE)
    return text

# Function to remove punctuation and digits
def remove_punct(text):
    text = replace_emojis(text)
    text = "".join([char for char in text if char not in string.punctuation])
    text = re.sub('[0-9]+', '', text)
    return text

# Apply cleaning
df['Tweet_punct'] = df['content'].apply(lambda x: remove_punct(x))

# Convert text to lowercase
df['lower'] = df['Tweet_punct'].apply(lambda x: x.lower())

# Tokenize text

```

```

df['Tokenized'] = df['lower'].apply(nltk.word_tokenize)

# Stopword removal
stopwords = nltk.corpus.stopwords.words('english')
df['no_stopwords'] = df['Tokenized'].apply(lambda x: [word for word in x if word not in
stopwords])

# Lemmatization
lemmatizer = nltk.WordNetLemmatizer()
df['Lemma'] = df['no_stopwords'].apply(lambda x: [lemmatizer.lemmatize(word) for word
in x])

# Join lemmatized words into text again
df['final'] = df['Lemma'].apply(lambda x: ' '.join(x))

# Vectorization
cv = CountVectorizer()
x = cv.fit_transform(df['final']).toarray()
y = df['sentiment']

# Split data
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.25, random_state=0)

# Train models
models = {
    "Logistic Regression": LogisticRegression(),
    "Naive Bayes": MultinomialNB(),
    "Random Forest": RandomForestClassifier(),
    "Support Vector Machine": SGDClassifier(loss='hinge', penalty='l2')
}

# Evaluate models
for name, model in models.items():
    print(f"Training {name}...")
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print(f"\n{name} Accuracy: {accuracy_score(y_test, y_pred):.4f}")
    print(f"{name} Classification Report:\n{classification_report(y_test, y_pred)}")
    ConfusionMatrixDisplay(confusion_matrix(y_test, y_pred),
display_labels=model.classes_).plot()
    plt.title(f"{name} - Confusion Matrix")
    plt.show()

```

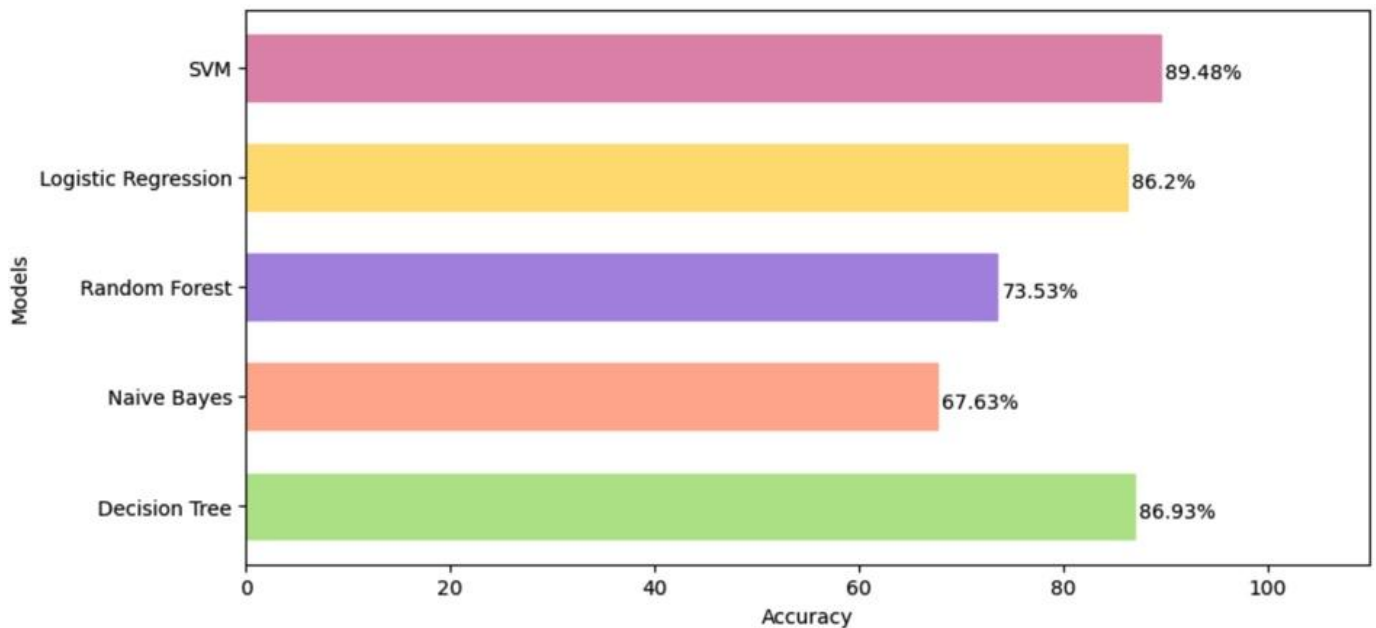
Chapter 5 : Results

In this chapter, we evaluate the performance of various machine learning models used for emotion detection from text. The models were trained and tested on the SemEval-2018 Affect in Tweets Dataset, which includes six emotion classes: **anger**, **fear**, **joy**, **love**, **sadness**, and **surprise**.

5.1 Accuracy

Accuracy is defined as the percentage of correct predictions for the test data. Here Accuracy of 5 models are shown. So that we can easily understand which model is performing better.

Accuracy of 5 models :



The accuracy of each model is summarized below:

- Support Vector Machine (SVM): 85.675%
- Logistic Regression: 80.847%
- Random Forest Classifier: 75.805%
- Multinomial Naive Bayes: 43.572%
- Decision Tree Classifier: 83.434%

SVM achieved the highest accuracy among all models, closely followed by Decision Tree.

5.2 Classification Report

The classification report includes **Precision**, **Recall**, and **F1-Score** for each emotion class. These metrics provide a detailed understanding of the model's ability to correctly identify each specific emotion.

Precision

Precision is the ratio of correctly predicted positive observations to the total predicted positives. It indicates how many selected items are relevant.

	SVM	Logistic Regression	Random Forest	Naive Bayes	Decision Tree
anger	90.549	90.704	77.675	94.649	88.265
fear	88.494	87.404	75.882	87.963	83.641
joy	88.832	82.406	78.812	63.378	90.984
love	84.028	86.197	68.269	100.000	75.337
sadness	91.619	88.403	74.973	67.402	88.560
surprise	86.145	93.333	70.952	100.000	70.213

Recall

Recall is the ratio of correctly predicted positive observations to all actual positives. It indicates how many relevant items are selected.

	SVM	Logistic Regression	Random Forest	Naive Bayes	Decision Tree
anger	85.628	77.778	71.014	34.179	85.386
fear	84.076	76.788	69.636	25.641	85.560
joy	94.552	95.344	77.563	97.028	86.974
love	75.000	63.223	73.347	7.025	80.785
sadness	94.344	94.227	80.700	92.595	90.729
surprise	67.136	52.582	69.953	1.878	77.465

F1 - Score

F1-Score is the weighted average of precision and recall. It is especially useful when the class distribution is imbalanced.

- SVM: 85.675%
- Logistic Regression: 80.847%
- Random Forest: 75.805%
- Naive Bayes: 43.572%
- Decision Tree: 83.434%

5.3 Confusion Matrix

The confusion matrix is a useful tool to visualize the performance of classification models by showing the actual vs. predicted classifications.

Confusion matrices for all 5 models (SVM, Logistic Regression, Random Forest, Naive Bayes, and Decision Tree) are provided in the below figures . These matrices show the correct predictions along the diagonal and misclassifications elsewhere.

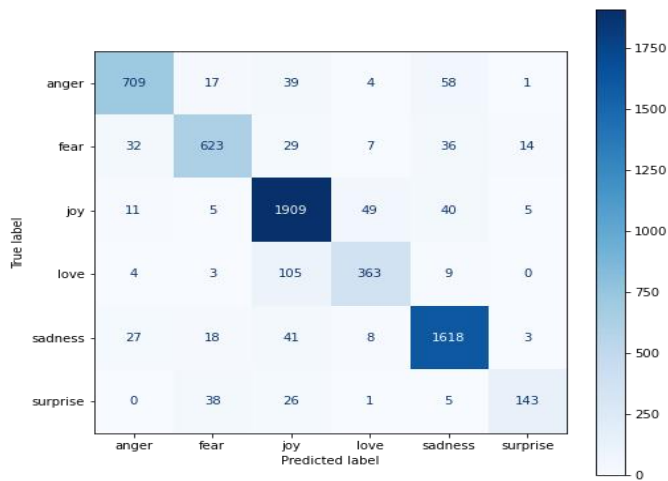


Figure i : SVM

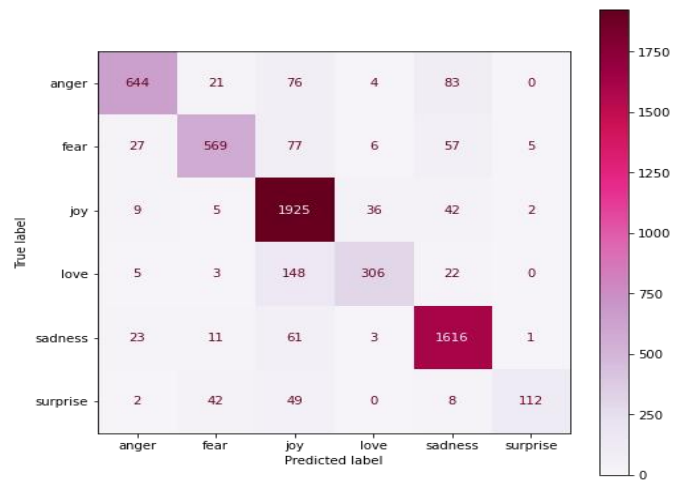


Figure ii : Logistic Regression

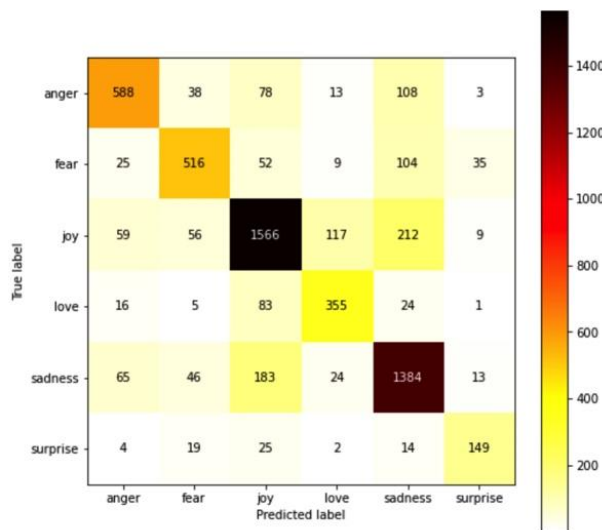


Figure iii : Random Forest

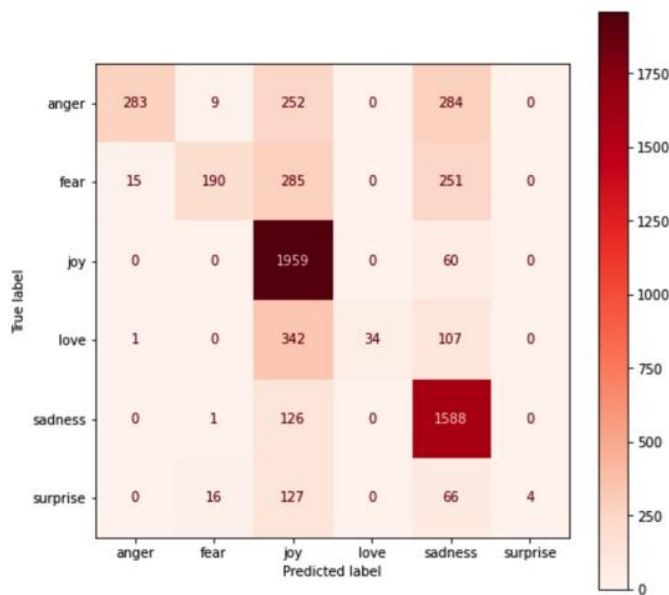


Figure iv : Multinomial NB

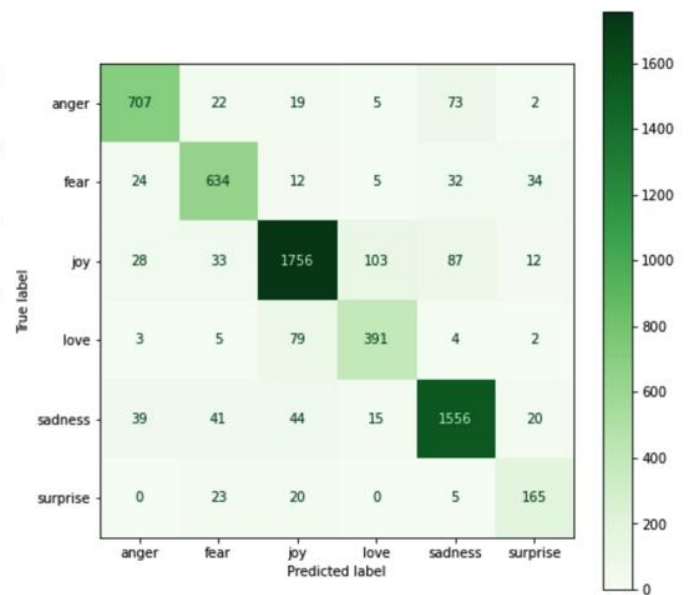


Figure v : Decision Tree

5.4 Summary

SVM provided the most balanced and accurate performance across all metrics.

Decision Tree also performed well and is a good alternative.

Logistic Regression showed strong recall but slightly lower precision.

Naive Bayes, although highly precise in some classes, struggled with recall, resulting in a low F1-score.

Random Forest showed moderate performance and could benefit from further tuning.

Overall, **SVM** is recommended as the best-performing model for emotion detection from text in this study.

Chapter 6 : Conclusion

The primary goal of this project was to detect human emotions from text using machine learning techniques. Understanding emotions in textual data is a crucial step in the development of emotionally intelligent systems, such as virtual assistants, social media monitors, customer support bots, and psychological support tools.

We utilized the SemEval-2018 Affect in Tweets dataset, which comprises over 20,000 labeled examples across six emotion categories: **anger**, **fear**, **joy**, **love**, **sadness**, and **surprise**. Due to the noisy nature of tweets and informal language, a significant portion of the work focused on effective **data preprocessing**, including:

Removal of punctuation and unnecessary symbols,

Tokenization and stopword removal,

Lemmatization to normalize word forms.

After cleaning and processing the data, we extracted features using both **Count Vectorization** and **TF-IDF**, which helped convert textual information into structured numerical formats required by machine learning algorithms.

We implemented five different classification models:

Support Vector Machine (SVM)

Logistic Regression

Random Forest Classifier

Multinomial Naive Bayes

Decision Tree Classifier

Performance Overview

Among the implemented models, **SVM** outperformed others in terms of accuracy and overall balance between **precision**, **recall**, and **F1-score**. It achieved an accuracy of **85.675%**, demonstrating its robustness in handling high-dimensional textual data. **Decision Tree** and **Logistic Regression** also yielded competitive results, offering alternative options based on simplicity and interpretability.

Although **Multinomial Naive Bayes** showed high precision for some emotion classes (especially 'love' and 'surprise'), it significantly underperformed in terms of recall. This reflects the model's inability to detect all relevant instances of those emotions, especially when the training data was imbalanced.

Challenges Encountered

Imbalanced Dataset: Some emotion categories (e.g., ‘surprise’ and ‘love’) were underrepresented, which led to skewed performance in certain models.

Ambiguity in Language: Human emotions are often expressed in subtle and ambiguous ways, making accurate classification a non-trivial task

Noisy Data: Tweets, being informal and unstructured, required extensive preprocessing to make them suitable for machine learning.

Key Contributions

A comparative evaluation of multiple classifiers for emotion detection.

Effective preprocessing pipeline tailored for social media text.

Visualization of performance metrics including confusion matrices, precision, recall, and F1-scores.

Demonstration of how classical machine learning models can be used effectively for emotion detection tasks.

Real-World Impact

Emotion detection has many applications in modern technology:

Sentiment Monitoring: Understanding public sentiment on social platforms.

Customer Experience Management: Analyzing feedback for service improvement.

Mental Health Tracking: Detecting emotional distress in user posts and messages.

Human-Computer Interaction: Enhancing interactions by making systems emotionally aware.

Final Remarks

This project lays the foundation for deeper exploration into affective computing and emotional AI. With further improvements—such as using **deep learning models (e.g., LSTM, BERT)**, incorporating **contextual embeddings**, and addressing **data imbalance**—this system can become even more powerful and accurate.

The encouraging results obtained show that emotion detection from text using machine learning is not only feasible but also impactful in enhancing the emotional intelligence of applications across various domains.

References

<https://www.kaggle.com/ishantjuyal/emotion>

Appendix

Emotion-detection-from-text

1 Emotion Detection from Text

Emotion is one of the basic instincts of a human being. Emotion detection plays a vital role in the field of textual analysis. At present, people's expressions and emotional states have turned into the leading topic for research works. Emotion Detection and Recognition from texts are recent fields of research that are closely related to Emotion Analysis. Emotion Analysis aims at detecting and recognizing feelings through the expressions from sentences, such as *anger, surprise, joy, fear, sadness, love* etc.

Course Name: Machine Learning Lab

Course Teachers

- B.Lalitha Rajeswari

Team Members

- 23BQ1A4219 - Bhuvanagiri Bhavani

```
[1]: # Basic Libraries
import pandas as pd
import numpy as np

# Visualization libraries
import seaborn as sns
import matplotlib.pyplot as plt

# Text Libraries
import nltk
import string
import re

# Feature Extraction Libraries
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.feature_extraction.text import TfidfTransformer
```

```

from sklearn.model_selection import train_test_split

# Classifier Model libraries
from sklearn.linear_model import SGDClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn import tree
# from sklearn.pipeline import Pipeline

# Performance Matrix libraries
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report
from sklearn.metrics import ConfusionMatrixDisplay

# other
import warnings
warnings.filterwarnings("ignore")

```

2 1. Dataset

```

[3]: df =
      pd.read_csv('text_emotions.csv') df

```

```

[3]                                     :      content sentiment
0                                i didnt feel humiliated      sadness
1      i can go from feeling so hopeless to so damned...      sadness
2      im grabbing a minute to post i feel greedy wrong      anger
3      i am ever feeling nostalgic about the fireplac...      love
4                                i am feeling grouchy      anger
...
19995 im having ssa examination tomorrow in the morn...      sadness
19996 i constantly worry about their fight against n...      joy
19997 i feel its important to share this info for th...      joy
19998 i truly feel that if you are passionate enough...      joy
19999 i feel like i just wanna buy any cute make up ...      joy

```

[20000 rows x 2 columns]

```

[4] print('Dataset size:',df.shape)
      print('Columns are:',df.columns)
      Y = df['sentiment']

```

Dataset size: (20000, 2)

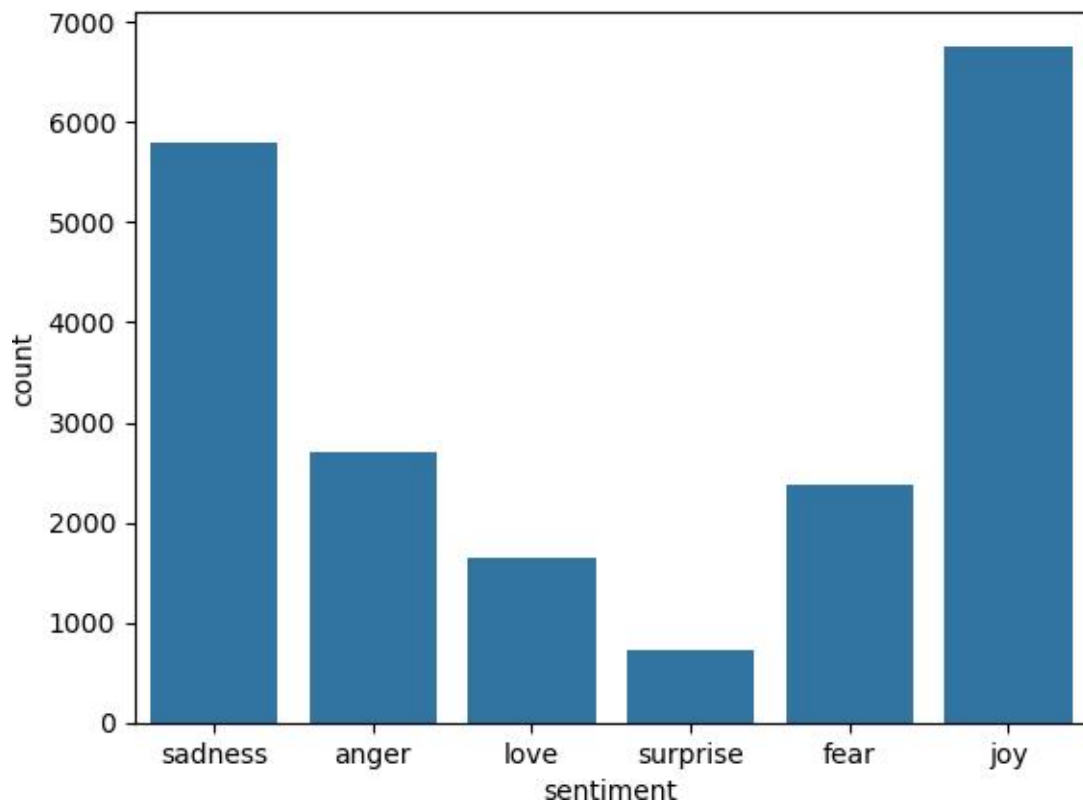
Columns are: Index(['content', 'sentiment'], dtype='object')

[5] df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   content     20000 non-null  object
1   sentiment   20000 non-null  object
dtypes: object(2)
memory usage: 312.6+ KB
```

[7]: **2.1 Dataset Emotion Analysis**

[7]: sns.countplot(x = 'sentiment', data = df)



3 2. Pre-processing text data

1. Remove punctuations
2. Tokenization - Converting a sentence into list of words
3. Remove stopwords
4. Lemmatization/stemming - Transforming any form of a word to its root word

3.1 2.1 Remove Punctuations

[10]:

```
emojis = pd.read_csv('emojis.txt',sep=',',header=None)
emojis_dict = {i:j for i,j in zip(emojis[0],emojis[1])}
pattern = ''.join(sorted(re.escape(k) for k in
emojis_dict))

def replace_emojis(text):
    text = re.sub(pattern,lambda m: emojis_dict.get(m.group(0)), text, flags=re.
IGNORECASE)
    return text

def remove_punct(text):
    text = replace_emojis(text)
    text = ''.join([char for char in text if char not in
string.punctuation]) text = re.sub('[0-9]+', '', text)
    return text

df['Tweet_punct'] = df['content'].apply(lambda x:
remove_punct(x)) df.head(10)
```

[10]:

	content	sentiment	\
0	i didnt feel humiliated	sadness	
1	i can go from feeling so hopeless to so damned...	sadness	
2	im grabbing a minute to post i feel greedy wrong	anger	
3	i am ever feeling nostalgic about the fireplac...	love	
4	i am feeling grouchy	anger	
5	ive been feeling a little burdened lately wasn...	sadness	
6	ive been taking or milligrams or times recomme...	surprise	
7	i feel as confused about life as a teenager or...	fear	
8	i have been with petronas for years i feel tha...	joy	
9	i feel romantic too	love	

	Tweet_punct
0	i didnt feel humiliated
1	i can go from feeling so hopeless to so damned...
2	im grabbing a minute to post i feel greedy wrong
3	i am ever feeling nostalgic about the fireplac...
4	i am feeling grouchy
5	ive been feeling a little burdened lately wasn...
6	ive been taking or milligrams or times recomme...
7	i feel as confused about life as a teenager or...

```

8 i have been with petronas for years i feel tha...
9 i feel romantic too

```

3.2 2.2 Tokenization

```

[12]: def tokenization(text):
      text = text.lower()
      text = re.split('\W+', text)

      return text

df['Tweet_tokenized'] = df['Tweet_punct'].apply(lambda x: tokenization(x.
lower()))
df.head(10)

```

```

[12]: \
      content sentiment \
0 i didnt feel humiliated sadness
1 i can go from feeling so hopeless to so damned... sadness
2 im grabbing a minute to post i feel greedy wrong anger
3 i am ever feeling nostalgic about the fireplac... love
4 i am feeling grouchy anger
5 ive been feeling a little burdened lately wasn... sadness
6 ive been taking or milligrams or times recomme... surprise
7 i feel as confused about life as a teenager or... fear
8 i have been with petronas for years i feel tha... joy
9 i feel romantic too love

```

```

      Tweet_punct \
0 i didnt feel humiliated
1 i can go from feeling so hopeless to so damned...
2 im grabbing a minute to post i feel greedy wrong
3 i am ever feeling nostalgic about the fireplac...
4 i am feeling grouchy
5 ive been feeling a little burdened lately wasn...
6 ive been taking or milligrams or times recomme...
7 i feel as confused about life as a teenager or...
8 i have been with petronas for years i feel tha...
9 i feel romantic too

```

```

      Tweet_tokenized
0 [i, didnt, feel, humiliated]
1 [i, can, go, from, feeling, so, hopeless, to, ...
2 [im, grabbing, a, minute, to, post, i, feel, g...
3 [i, am, ever, feeling, nostalgic, about, the, ...
4 [i, am, feeling, grouchy]
5 [ive, been, feeling, a, little, burdened, late...
6 [ive, been, taking, or, milligrams, or, times,...

```

```

7 [i, feel, as, confused, about, life, as, a, te...
8 [i, have, been, with, petronas, for, years, i,...
9 [i, feel, romantic, too]

```

3.3 2.3 Remove Stopwords

```

[14]: # Install and configure NLTK downloads (bypass SSL verification if
      # needed)
      import ssl
      import nltk

      # Disable SSL verification temporarily for nltk downloader if SSL error
      # exists
      try:
          _create_unverified_https_context = ssl._create_unverified_context
      except AttributeError:
          pass
      else:
          ssl._create_default_https_context = _create_unverified_https_context

      # Download required NLTK
      #resources nltk.download('stopwords')
      nltk.download('wordnet')

      # Import stopwords and WordNet lemmatizer
      from nltk.corpus import stopwords
      from nltk.stem import WordNetLemmatizer

      # Load English stopwords
      stopword = stopwords.words('english')

      # Add your custom stopwords
      custom_stopwords = [
          'yr', 'year', 'woman', 'man', 'girl', 'boy', 'one', 'two', 'sixteen', '_
          'yearold',
          'fu', 'weeks', 'week', 'treatment', 'associated', 'patients', 'may', 'day', '_
          'case',
          'old', 'u', 'n', 'didnt', 'ive', 'ate', 'feel', 'keep', 'brother', 'dad', '_
          'basic', 'im'

```

```
]
stopword.extend(custom_stopwords)
```

```
[15]: def remove_stopwords(text):
      text = [word for word in text if word not in stopword]
      return text

df['Tweet_nonstop'] = df['Tweet_tokenized'].apply(lambda x:
remove_stopwords(x)) df.head(10)
```

```
[15]:                                     content sentiment \
0                                     i didnt feel humiliated    sadness
1    i can go from feeling so hopeless to so damned...    sadness
2    im grabbing a minute to post i feel greedy wrong        anger
3    i am ever feeling nostalgic about the fireplac...        love
4                                     i am feeling grouchy        anger
5    ive been feeling a little burdened lately wasn...    sadness
6    ive been taking or milligrams or times recomme...    surprise
7    i feel as confused about life as a teenager or...        fear
8    i have been with petronas for years i feel tha...        joy
9                                     i feel romantic too        love
```

```
                                     Tweet_punct \
0                                     i didnt feel humiliated
1    i can go from feeling so hopeless to so damned...
2    im grabbing a minute to post i feel greedy wrong
3    i am ever feeling nostalgic about the fireplac...
4                                     i am feeling grouchy
5    ive been feeling a little burdened lately wasn...
6    ive been taking or milligrams or times recomme...
7    i feel as confused about life as a teenager or...
8    i have been with petronas for years i feel tha...
9                                     i feel romantic too
```

```
                                     Tweet_tokenized \
0                                     [i, didnt, feel, humiliated]
1    [i, can, go, from, feeling, so, hopeless, to, ...
2    [im, grabbing, a, minute, to, post, i, feel, g...
3    [i, am, ever, feeling, nostalgic, about, the, ...
4                                     [i, am, feeling, grouchy]
5    [ive, been, feeling, a, little, burdened, late...
6    [ive, been, taking, or, milligrams, or, times,...
7    [i, feel, as, confused, about, life, as, a, te...
8    [i, have, been, with, petronas, for, years, i,...
9                                     [i, feel, romantic, too]
```

```
                                     Tweet_nonstop
0                                     [humiliated]
1    [go, feeling, hopeless, damned, hopeful, aroun...
2    [grabbing, minute, post, greedy, wrong]
```

```

3 [ever, feeling, nostalgic, fireplace, know, st...
4                                     [feeling, grouchy]
5 [feeling, little, burdened, lately, wasnt, sure]
6 [taking, milligrams, times, recommended, amoun...
7                                     [confused, life, teenager, jaded]
8 [petronas, years, petronas, performed, well, m...
9                                     [romantic]

```

3.4 2.4 Lemmatization

```

[17]: nltk.download('wordnet')
      wn = nltk.WordNetLemmatizer()

      def lemmatizer(text):
          text = [wn.lemmatize(word) for word in text]
          return text

      df['Tweet_lemmatized'] = df['Tweet_nonstop'].apply(lambda x: lemmatizer(x))
      df.head()

[nltk_data] Downloading package wordnet to
[nltk_data] /Users/lakshmivinay/nltk_data...
[nltk_data] Package wordnet is already up-to-date!

```

```

[17]:                                     content sentiment \
0                                     i didnt feel humiliated sadness
1 i can go from feeling so hopeless to so damned... sadness
2 im grabbing a minute to post i feel greedy wrong anger
3 i am ever feeling nostalgic about the fireplac... love
4                                     i am feeling grouchy anger

                                     Tweet_punct \
0                                     i didnt feel humiliated
1 i can go from feeling so hopeless to so damned...
2 im grabbing a minute to post i feel greedy wrong
3 i am ever feeling nostalgic about the fireplac...
4                                     i am feeling grouchy

                                     Tweet_tokenized \
0                                     [i, didnt, feel, humiliated]
1 [i, can, go, from, feeling, so, hopeless, to, ...
2 [im, grabbing, a, minute, to, post, i, feel, g...
3 [i, am, ever, feeling, nostalgic, about, the, ...
4                                     [i, am, feeling, grouchy]

                                     Tweet_nonstop \
0                                     [humiliated]

```

```

1 [go, feeling, hopeless, damned, hopeful, aroun...
2     [grabbing, minute, post, greedy, wrong]
3 [ever, feeling, nostalgic, fireplace, know, st...
4     [feeling, grouchy]

                                Tweet_lemmatized
0                                [humiliated]
1 [go, feeling, hopeless, damned, hopeful, aroun...
2     [grabbing, minute, post, greedy, wrong]
3 [ever, feeling, nostalgic, fireplace, know, st...
4     [feeling, grouchy]

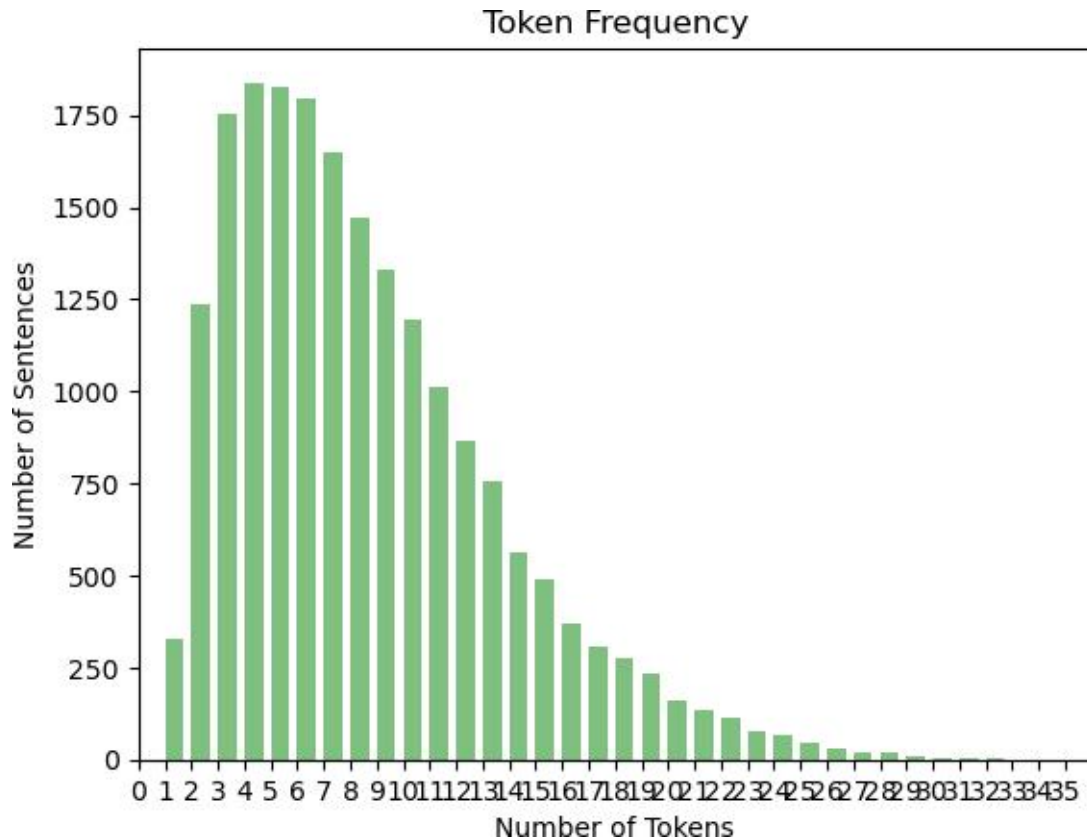
```

3.5 Token Frequency in a sentence after preprocessing

```

[19]: token_lengths = [len(t) for t in df['Tweet_lemmatized']]
plt.hist(token_lengths, bins=33, width = 0.7, fc="green", alpha=0.5)
plt.xlabel('Number of Tokens')
plt.ylabel('Number of Sentences')
plt.title('Token Frequency')
x_Values = [i for i in
range(36)]
plt.xticks(x_Values)
plt.xlim(0,36)
plt.rcParams["figure.figsize"] = (10,5)
plt.show()

```



3.6 All in one (Text preprocessing)

```
[21]: def clean_text(text):
    text = remove_punct(text)
    text = tokenization(text)
    text =
    remove_stopwords(text) text
    = lemmatizer(text) return
    text
```

4 3. Feature Extraction

1. Count Vectorizer
2. TF-IDF (Term Frequency and Inverse Document Frequency)

4.1 3.1 Count Vectorizer

CountVectorizer is a great tool provided by the *scikit-learn* library in Python. It is used to transform a given text into a vector on the basis of the frequency (count) of each word that occurs in the entire text. This is helpful when we have multiple such texts, and we wish to convert each

word in each text into vectors (for using in further text analysis).

Let us consider a few sample texts from a document:

document 1: Time flies flies like an arrow.

document 2: Fruit flies like a banana.

CountVectorizer creates a matrix in which each unique word is represented by a column of the matrix, and each text sample from the document is a row in the matrix. The value of each cell is nothing but the count of the word in that particular text sample.

```
[28]: document = ["Time flies flies like an arrow",
                  "Fruit flies like a banana"]

# Create a Vectorizer Object
vectorizer = CountVectorizer()

vectorizer.fit(document)

# Printing the identified Unique words along with their indices
print("Vocabulary: ", vectorizer.vocabulary_)

# Encode the Document
vector = vectorizer.transform(document)

# Summarizing the Encoded Texts
print("Encoded Document is:")
print(vector.toarray())

dictionary = sorted(vectorizer.vocabulary_)

print(dictionary)

sns.heatmap(vector.toarray(), annot=True, cbar=False, xticklabels=dictionary,
            yticklabels=['Sentence_1', 'Sentence 2'])
```

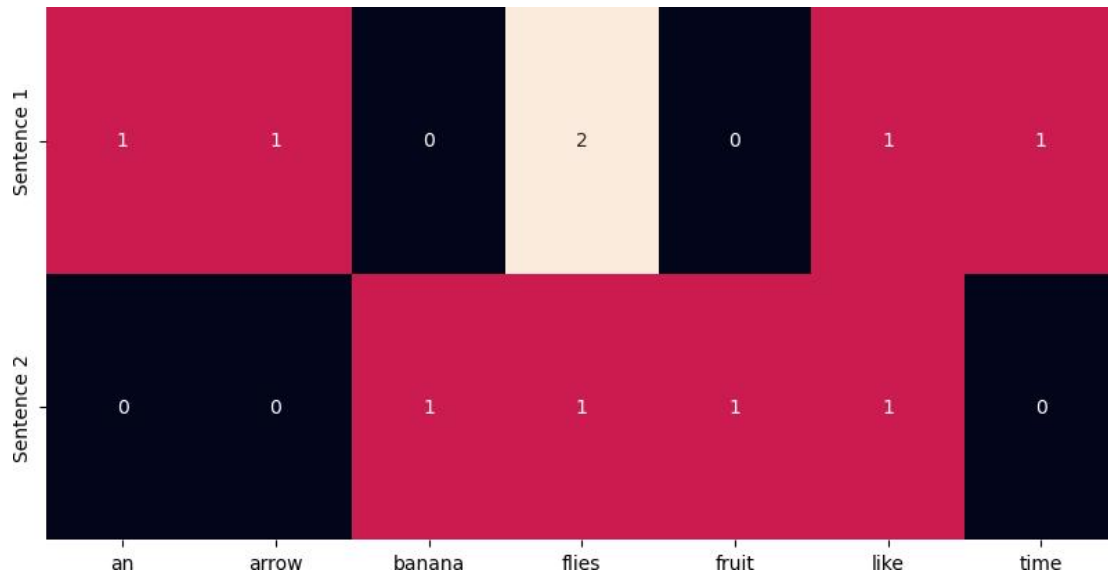
Vocabulary: {'time': 6, 'flies': 3, 'like': 5, 'an': 0, 'arrow': 1, 'fruit': 4, 'banana': 2}

Encoded Document is:

```
[[1 1 0 2 0 1 1]
 [0 0 1 1 1 1 0]]
```

['an', 'arrow', 'banana', 'flies', 'fruit', 'like', 'time']

```
[28]: <Axes: >
```



4.2 3.2 TF-IDF

4.2.1 3.2.1 Term Frequency (TF)

Term Frequency (TF) is a measure of how frequently a term, t , appears in a document, d :

$$TF_{t,d} = \frac{w_{t,d}}{\text{Total number of terms in document } d}$$

$w_{t,d}$ = Number of times term t appears in a document d . Thus, each document and term would have its own TF value.

Consider these 3 documents:-

- It was the best of the time.
- it was the worst of Times.
- it is the time of stupidity.

The vocabulary or dictionary of the entire corpus would be:-

1. it
2. was
3. the
4. best
5. of
6. time
7. worst
8. is
9. stupidity

Now we will calculate the TF values for the **Document 3**.

Document 3 :- **it is the time of stupidity.**

- Number of words in Document 3 = **6**
- TF for the word **‘the’** = (number of times **‘the’** appears in Document 3) / (number of terms in Document 3) = **1/6**

Likewise:-

- TF(**‘it’**) = 1/6
- TF(**‘was’**) = 0/6 = 0
- TF(**‘the’**) = 1/6
- TF(**‘best’**) = 0/6 = 0
- TF(**‘of’**) = 1/6
- TF(**‘time’**) = 1/6
- TF(**‘worst’**) = 0/6 = 0
- TF(**‘is’**) = 1/6
- TF(**‘stupidity’**) = 1/6

We can calculate all the term frequencies for all the terms of all the documents in this manner:-

Term	Document#1	Document#2	Document#3	TF (Document#1)	TF (Document#2)	TF (Document#3)
it	1	1	1	1/7	1/6	1/6
was	1	1	0	1/7	1/6	0
the	2	1	1	2/7	1/6	1/6
best	1	0	0	1/7	0	0
of	1	1	1	1/7	1/6	1/6
time	1	1	1	1/7	1/6	1/6
worst	0	1	0	0	1/6	0
is	0	0	1	0	0	1/6
stupidity	0	0	1	0	0	1/6

4.2.2 3.2.2 Inverse Document Frequency (IDF)

IDF is a measure of how important a term is. We need the IDF value because computing just the **TF alone is not sufficient** to understand the importance of words:

$$\text{IDF}_{\text{term}} = \frac{1}{\log \left(\frac{\text{Total Number of Documents}}{\text{The Number of Documents with Term}} \right)}$$

A problem with scoring word frequency is that highly frequent words (**‘is’, ‘the’, ‘a’ etc**) start to dominate in the document (e.g. larger score), but may not contain as much **“useful information”** to the model compared to the rarer but **domain specific words**.

One approach is to rescale the frequency of words by **how often they appear in all documents**, so that the scores for frequent words like “the” that are also frequent **across all documents are penalized**.

This approach to scoring is called Term Frequency – Inverse Document Frequency, or TF-IDF for short, where:

- **Term Frequency:** is a scoring of the frequency of the word in the current document.
- **Inverse Document Frequency:** is a scoring of how rare the word is across documents.

Thus the idf of a rare term is high, whereas the idf of a frequent term is likely to be low.

We can calculate the IDF values for **Document 3**:

Document 3 :- **it is the time of stupidity.**

$IDF('it') = \log(\text{total number of documents} / \text{number of documents containing the word 'it'}) = \log(3/3) = \log(1) = 0$

$IDF('stupidity') = \log(\text{total number of documents} / \text{number of documents containing the word 'stupidity'}) = \log(3/1) = \log(3) = 0.48$

We can calculate the IDF values for each word like this. Thus, the IDF values for the entire vocabulary would be:

Term	Document#1	Document#2	Document#3	IDF
it	1	1	1	0.00
was	1	1	0	0.18
the	2	1	1	0.00
best	1	0	0	0.48
of	1	1	1	0.00
time	1	1	1	0.00
worst	0	1	0	0.48
is	0	0	1	0.48
stupidity	0	0	1	0.48

We can now compute the TF-IDF score for each word in the corpus. Words with a higher score are more important, and those with a lower score are less important:

$$(\text{TF} - \text{ឆ្លើយ})_{\text{ឆ្លើយ}, \text{ឆ្លើយ}} = \text{TF}_{\text{ឆ្លើយ}, \text{ឆ្លើយ}} * \text{ឆ្លើយ}_{\text{ឆ្លើយ}}$$

You can find the overall summary in the following figure.

We can now calculate the TF-IDF score for every word in **Document 3**:

Document 3 :- **it is the time of stupidity.**

$TF-IDF('it', \text{Document 3}) = TF('it', \text{Document 3}) * IDF('it') = 1/6 * 0 = 0$

Likewise:-

- $TF('it') = (1/6) * 0 = 0$
- $TF('is') = (1/6) * 0.48 = 0.08$
- $TF('the') = (1/6) * 0 = 0$
- $TF('best') = (0/6) * 0.48 = 0$
- $TF('time') = (1/6) * 0 = 0$
- $TF('of') = (1/6) * 0 = 0$

- $TF(\text{'stupidity'}) = (1/6) * 0.48 = 0.08$

Similarly, we can calculate the TF-IDF scores for all the words with respect to all the documents.

- First, notice how if there is a very common word that occurs in all documents (i.e., $n = N$), $IDF(w)$ is 0 and the TFIDF score is 0, thereby completely penalizing that term.
- Second, if a term occurs very rarely, perhaps in only one document, the IDF will be the maximum possible value, $\log N$

```
[35]: from sklearn.feature_extraction.text import TfidfVectorizer

document = ['Time flies flies like an arrow.',
            'Fruit flies like a banana.']

tfidf_vectorizer = TfidfVectorizer()
tfidf = tfidf_vectorizer.fit_transform(document).toarray()

print (tfidf)

print (tfidf_vectorizer.vocabulary_)

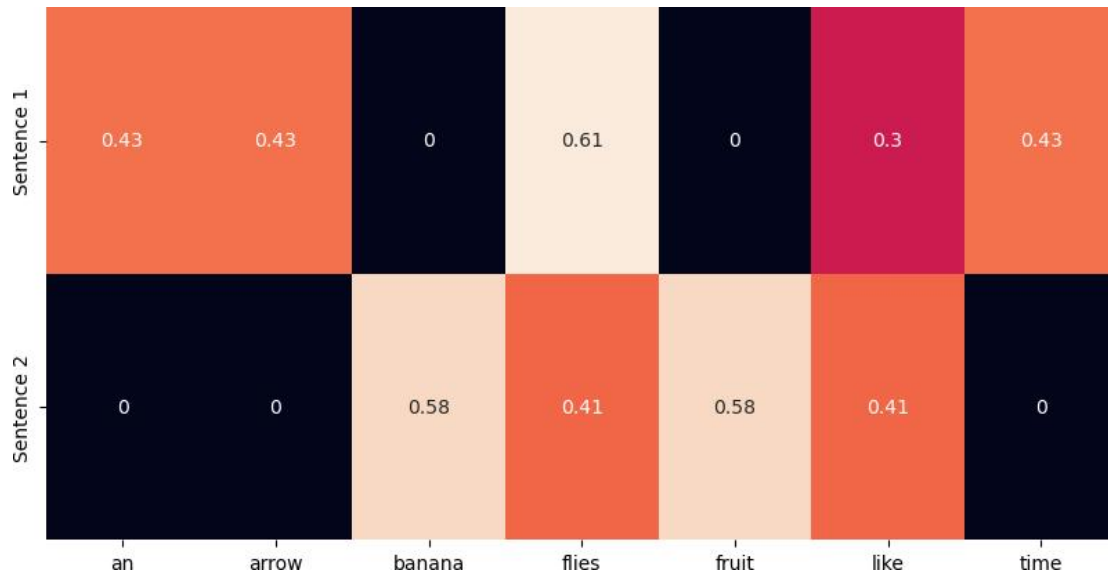
dictionary = sorted(tfidf_vectorizer.vocabulary_)

print(dictionary)

sns.heatmap(tfidf, annot=True, cbar=False, xticklabels=dictionary,
            yticklabels=['Sentence
1','Sentence_
2'])
```

```
[[0.42519636      0.          0.60506143  0.          0.3025307
0.42519636
0.42519636]
[0.          0.          0.5761523  0.40993715 0.57615236
0.          0.40993715
0.          ]]
{'time': 6, 'flies': 3, 'like': 5, 'an': 0, 'arrow': 1, 'fruit': 4, 'banana': 2}
['an', 'arrow', 'banana', 'flies', 'fruit', 'like', 'time']
```

[35]: <Axes: >



4.2.3 Applying Feature Extraction in tweet dataset Train

Test Split (70-30)

```
[38]: X_train, X_test, y_train, y_test = train_test_split(df['content'],_
    df['sentiment'],test_size=0.3, random_state = 0)
```

```
[39]: countVectorizer1 = CountVectorizer(analyzer=clean_text)
countVector1 = countVectorizer1.fit_transform(X_train)

countVector2 = countVectorizer1.transform(X_test)

tfidf_transformer_xtrain = TfidfTransformer()
x_train = tfidf_transformer_xtrain.fit_transform(countVector1)

tfidf_transformer_xtest = TfidfTransformer()
x_test = tfidf_transformer_xtest.fit_transform(countVector2)
```

5 4. Models

1. Support Vector Machine (SVM)
2. Logistic Regression
3. Random Forest Classifier
4. Multinomial Naive Bayes
5. Decision Tree Classifier

Performance Matrix :

1. **Accuracy** = $\frac{TP+TN}{TP+TN+FP+FN}$
2. **Precision** = $\frac{TP}{TP+FP}$
3. **Recall** = $\frac{TP}{TP+FN}$
4. **F1-score** = $\frac{2*Precision*Recall}{Precision+Recall}$

5.1 4.1 Support Vector Machine (SVM)

```
[43]: svm = SGDClassifier()
      svm.fit(x_train, y_train)

      y_pred = svm.predict(x_test)

      svm_acc = accuracy_score(y_pred, y_test)
      svm_prec = precision_score(y_test, y_pred, average='macro')
      svm_recal = recall_score(y_test, y_pred, average='macro')
      svm_cm = confusion_matrix(y_test, y_pred)
      svm_f1 = f1_score(y_test, y_pred, average='macro')

      print('Accuracy:',
            '{0:.3f}'.format(svm_acc*100))
      print('Precision:',
            '{0:.3f}'.format(svm_prec*100)) print('Recall:',
            '{0:.3f}'.format(svm_recal*100))
      print('F1-score:', '{0:.3f}'.format(svm_f1*100))
      print(classification_report(y_test, y_pred))
```

Precision: 88.420

Recall: 83.370

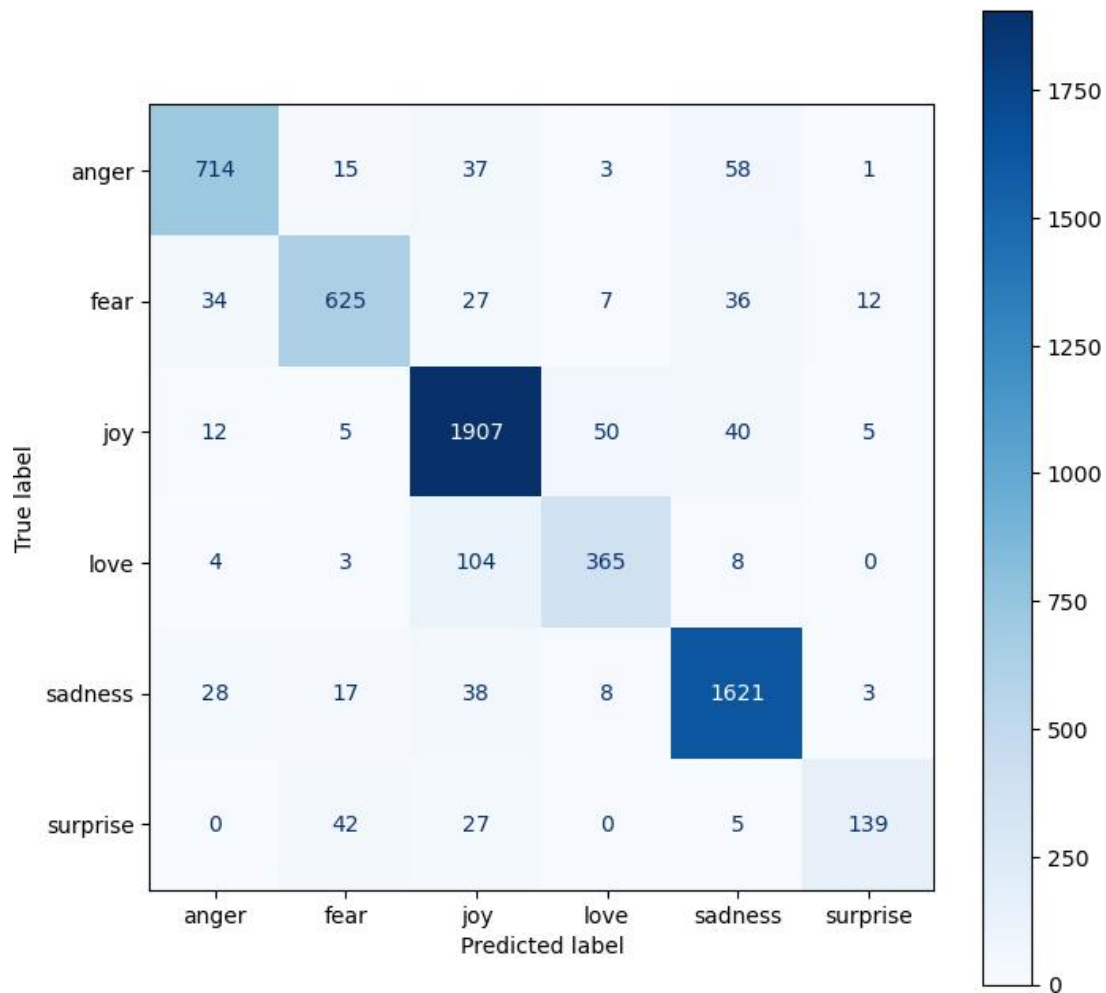
F1-score: 85.566

	precision	recall	f1-score	support
anger	0.90	0.86	0.88	828
fear	0.88	0.84	0.86	741
joy	0.89	0.94	0.92	2019
love	0.84	0.75	0.80	484
sadness	0.92	0.95	0.93	1715
surprise	0.87	0.65	0.75	213
accuracy			0.90	6000
macro avg	0.88	0.83	0.86	6000
weighted avg	0.89	0.90	0.89	6000

5.1.1 4.1.1 Confusion Matrix

```
[45]: cm_display_svm = ConfusionMatrixDisplay(svm_cm, display_labels=svm.classes_)
      fig, ax = plt.subplots(figsize=(8,8)) # adjust the size
      cm_display_svm.plot(ax=ax, cmap='Blues')
```

```
[45]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x154043080>
```



5.2 4.2 Logistic Regression

```
[47]: logisticRegr = LogisticRegression()
logisticRegr.fit(x_train, y_train)
y_pred = logisticRegr.predict(x_test)
lr_acc = accuracy_score(y_pred, y_test)
lr_prec = precision_score(y_test, y_pred, average='macro')
lr_recal = recall_score(y_test, y_pred, average='macro')
lr_cm = confusion_matrix(y_test, y_pred)
lr_f1 = f1_score(y_test, y_pred, average='macro')
print('Accuracy:', '{0:.3f}'.format(lr_acc*100))
```

```
print('Precision:', '{0:.3f}'.format(lr_prec*100))
print('Recall:', '{0:.3f}'.format(lr_recal*100))
print('F1-score:', '{0:.3f}'.format(lr_f1*100))
print(classification_report(y_test,y_pred))
```

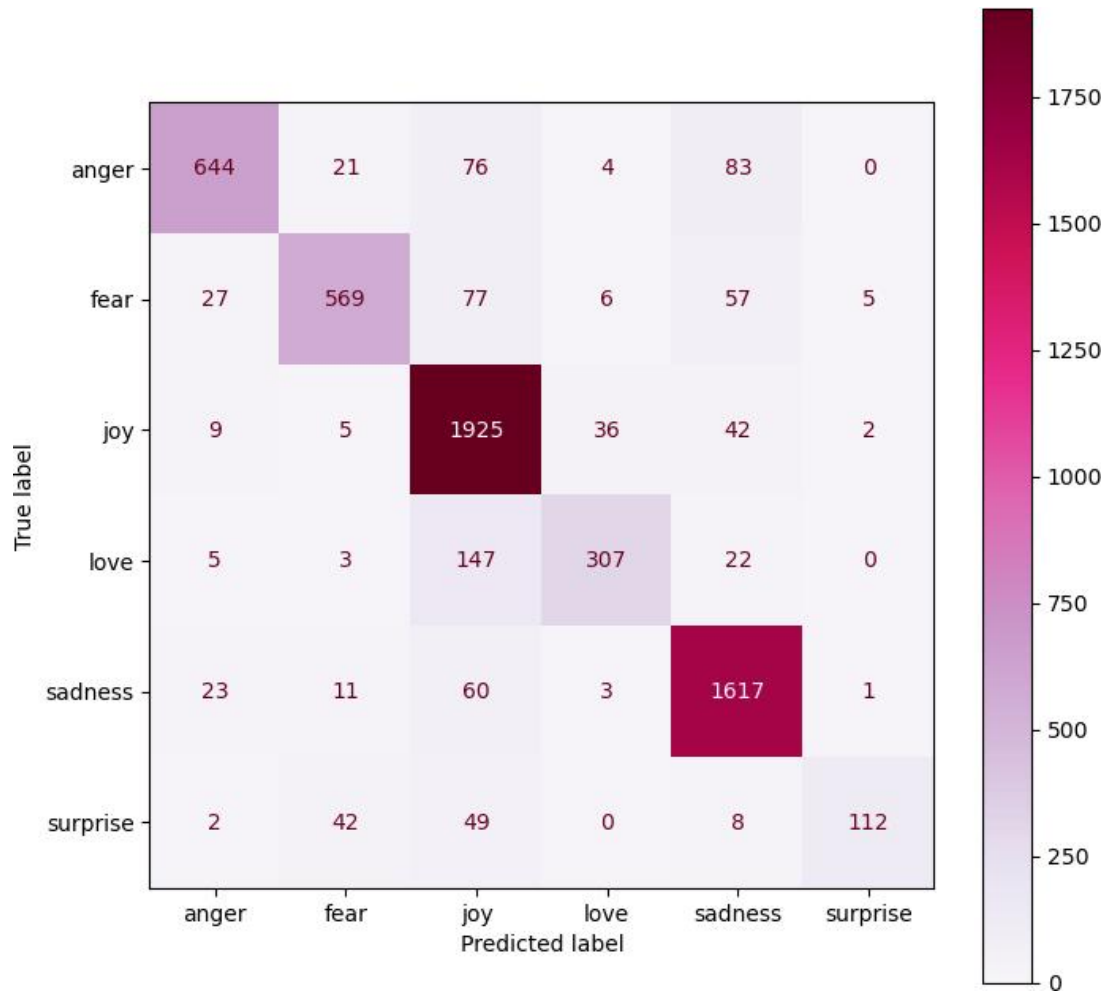
Accuracy: 86.233
Precision: 88.094
Recall: 76.701
F1-score: 80.926

	precision	recall	f1-score	support
anger	0.91	0.78	0.84	828
fear	0.87	0.77	0.82	741
joy	0.82	0.95	0.88	2019
love	0.86	0.63	0.73	484
sadness	0.88	0.94	0.91	1715
surprise	0.93	0.53	0.67	213
accuracy			0.86	6000
macro avg	0.88	0.77	0.81	6000
weighted avg	0.87	0.86	0.86	6000

5.2.1 4.2.1 Confusion Matrix

```
[49]: cm_display_lr = ConfusionMatrixDisplay(lr_cm, display_labels=logisticRegr.
      classes_)
      fig, ax = plt.subplots(figsize=(8,8)) # adjust the size
      cm_display_lr.plot(ax=ax,cmap='PuRd')
```

```
[49]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1542c8a40>
```



5.3 4.3 Random Forest Classifier

random forest builds multiple decision trees and merges them together to get a more accurate and stable prediction.

Hyperparameters:

Increasing the predictive power

Firstly, there is the **n_estimators** hyperparameter, which is just the number of trees the algorithm builds before taking the maximum voting or taking the averages of predictions. In general, a higher number of trees increases the performance and makes the predictions more stable, but it also slows down the computation.

Another important hyperparameter is **max_features**, which is the maximum number of features random forest considers to split a node. Sklearn provides several options, all described in the documentation.

The last important hyperparameter is **min_sample_leaf**. This determines the minimum number of leafs required to split an internal node.

[click for more](#)

```
[51]: rfc = RandomForestClassifier(n_estimators=1, random_state=0)

rfc.fit(x_train, y_train)

y_pred = rfc.predict(x_test)

rfc_acc = accuracy_score(y_test, y_pred)
rfc_prec = precision_score(y_test, y_pred, average='macro')
rfc_recal = recall_score(y_test, y_pred, average='macro')
rfc_cm = confusion_matrix(y_test, y_pred)
rfc_f1 = f1_score(y_test, y_pred, average='macro')

print('Accuracy:', '{0:.3f}'.format(rfc_acc*100))
print('Precision:', '{0:.3f}'.format(rfc_prec*100))
print('Recall:', '{0:.3f}'.format(rfc_recal*100))
print('F1-score:', '{0:.3f}'.format(rfc_f1*100))
print(classification_report(y_test, y_pred))
```

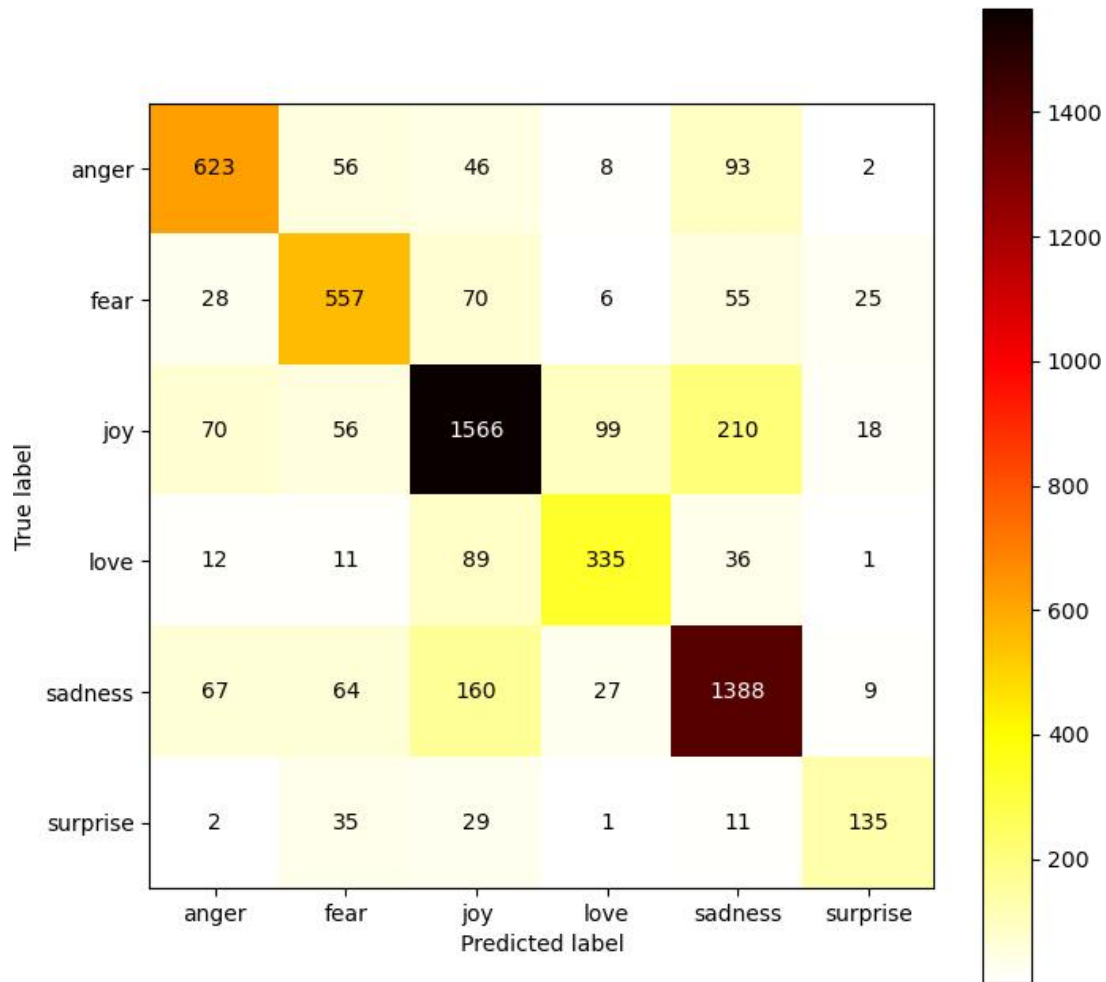
Accuracy: 76.733
Precision: 74.654
Recall: 73.584
F1-score: 74.061

	precision	recall	f1-score	support
anger	0.78	0.75	0.76	828
fear	0.72	0.75	0.73	741
joy	0.80	0.78	0.79	2019
love	0.70	0.69	0.70	484
sadness	0.77	0.81	0.79	1715
surprise	0.71	0.63	0.67	213
accuracy			0.77	6000
macro avg	0.75	0.74	0.74	6000
weighted avg	0.77	0.77	0.77	6000

5.3.1 4.3.1 Confusion Matrix

```
[53]: cm_display_rfc = ConfusionMatrixDisplay(rfc_cm, display_labels=rfc.classes_)
fig, ax = plt.subplots(figsize=(8,8)) # adjust the size
cm_display_rfc.plot(ax=ax, cmap='hot_r')
```

```
[53]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x155202c30>
```



5.4 45 Multinomial Naïve Bayes Classifier

```
[55]: mnb = MultinomialNB()
      mnb.fit(x_train, y_train)

      y_pred = mnb.predict(x_test)

      mnb_acc = accuracy_score(y_pred, y_test)
      mnb_prec = precision_score(y_test, y_pred, average='macro')
      mnb_recal = recall_score(y_test, y_pred, average='macro')
      mnb_cm = confusion_matrix(y_test, y_pred)
      mnb_f1 = f1_score(y_test, y_pred, average='macro')

      print('Accuracy:', '{0:.3f}'.format(mnb_acc*100))
      print('Precision:', '{0:.3f}'.format(mnb_prec*100))
      print('Recall:', '{0:.3f}'.format(mnb_recal*100))
```

```
print('F1-score:', '{0:.3f}'.format(mnb_f1*100))
print(classification_report(y_test,y_pred))
```

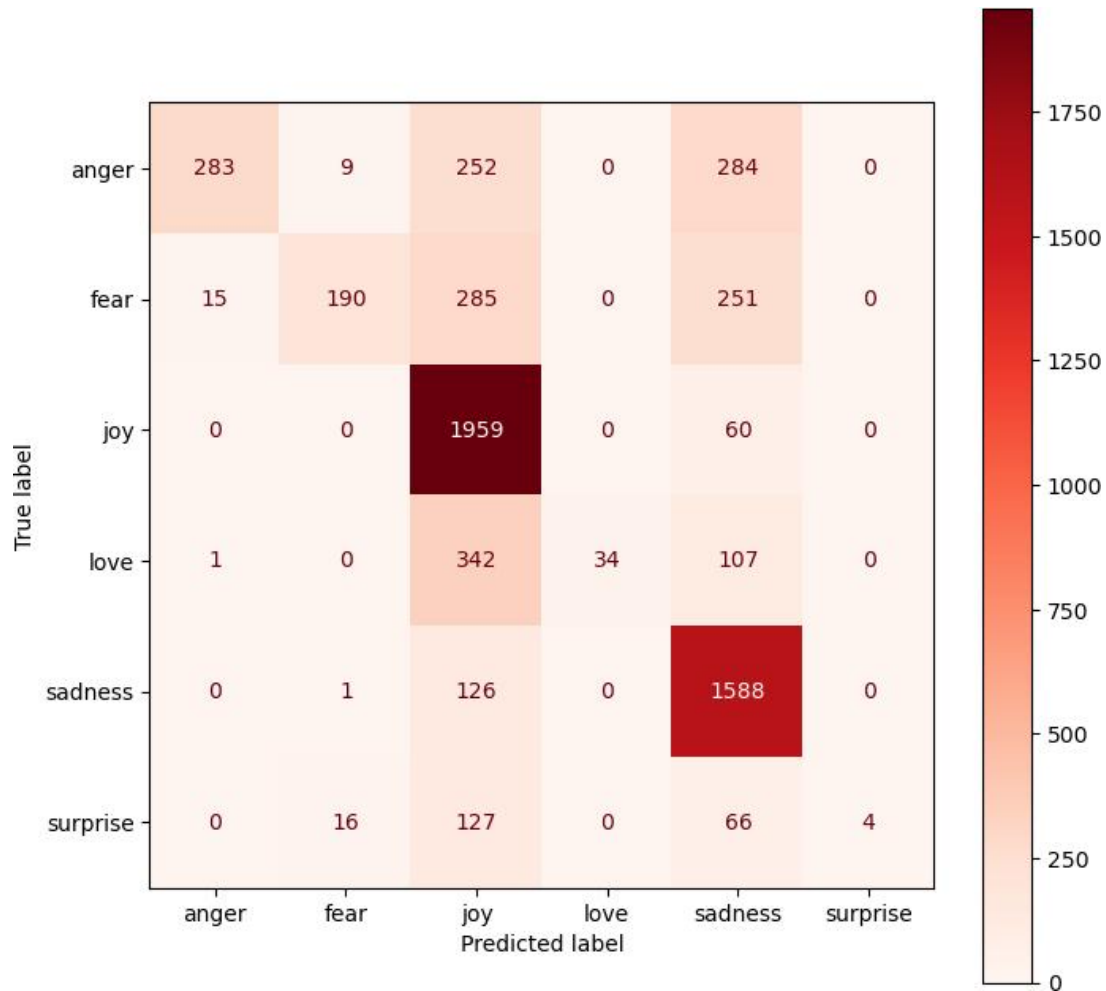
Accuracy: 67.633
Precision: 85.565
Recall: 43.058
F1-score: 43.572

	precision	recall	f1-score	support
anger	0.95	0.34	0.50	828
fear	0.88	0.26	0.40	741
joy	0.63	0.97	0.77	2019
love	1.00	0.07	0.13	484
sadness	0.67	0.93	0.78	1715
surprise	1.00	0.02	0.04	213
accuracy			0.68	6000
macro avg	0.86	0.43	0.44	6000
weighted avg	0.76	0.68	0.61	6000

5.4.1 4-5.1 Confusion Matrix

```
[57]: cm_display_mnb = ConfusionMatrixDisplay(mnb_cm, display_labels=mnb.classes_)
fig, ax = plt.subplots(figsize=(8,8)) # adjust the size
cm_display_mnb.plot(ax=ax,cmap='Reds')
```

```
[57]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1553579e0>
```



5.5 4.6 Decision Tree Classifier

```
[59]: dt = tree.DecisionTreeClassifier()
      dt.fit(x_train, y_train)
      y_pred = dt.predict(x_test)

      dt_acc = accuracy_score(y_pred, y_test)
      dt_prec = precision_score(y_test, y_pred, average='macro')
      dt_recal = recall_score(y_test, y_pred, average='macro')
      dt_cm = confusion_matrix(y_test, y_pred)
      dt_f1 = f1_score(y_test, y_pred, average='macro')

      print('Accuracy:', '{0:.3f}'.format(dt_acc*100))
      print('Precision:', '{0:.3f}'.format(dt_prec*100))
```

```
print('Recall:', '{0:.3f}'.format(dt_recal*100))
print('F1-score:', '{0:.3f}'.format(dt_f1*100))
print(classification_report(y_test,y_pred))
```

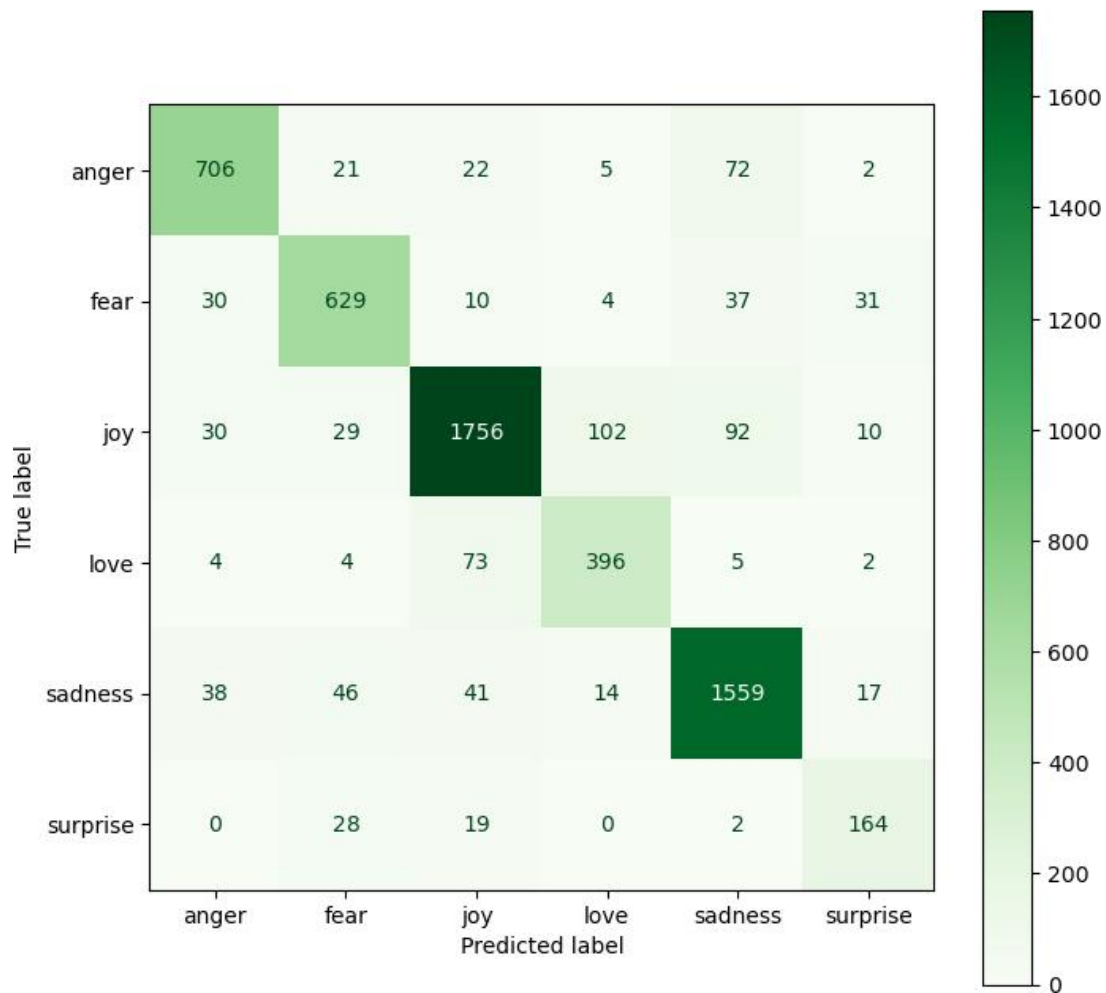
Accuracy: 86.833
Precision: 83.113
Recall: 84.474
F1-score: 83.749

	precision	recall	f1-score	support
anger	0.87	0.85	0.86	828
fear	0.83	0.85	0.84	741
joy	0.91	0.87	0.89	2019
love	0.76	0.82	0.79	484
sadness	0.88	0.91	0.90	1715
surprise	0.73	0.77	0.75	213
accuracy			0.87	6000
macro avg	0.83	0.84	0.84	6000
weighted avg	0.87	0.87	0.87	6000

5.5.1 4.6.1 Confusion Matrix

```
[61]: cm_display_dt = ConfusionMatrixDisplay(dt_cm, display_labels=dt.classes_)
fig, ax = plt.subplots(figsize=(8,8)) # adjust the size
cm_display_dt.plot(ax=ax,cmap='Greens')
```

```
[61]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1552f2960>
```



6 5. Summary Visualization

```
[63]: def get_prec_recall_per_emotion(cm):
    per_emotion_precision = []
    for i in range(len(cm)):
        count_per_emotion, accurate = 0, 0
        for j in range(len(cm)):
            if i == j:
                accurate = cm[j][i]
                count_per_emotion += cm[j][i]
        per_emotion_precision.append(round((accurate/count_per_emotion)*100, 3))

    per_emotion_recall = []
    for i in range(len(cm)):
        count_per_emotion, accurate = 0, 0
```

```

for j in range(len(cm)):
    if i == j:
        accurate = cm[i][j]
        count_per_emotion += cm[i][j]
    per_emotion_recall.append(round((accurate/count_per_emotion)*100,3))

return per_emotion_precision, per_emotion_recall

```

6.1 5.1 - Precision of all models

```

[65]: svm_per_prec,svm_per_recall =
get_prec_recall_per_emotion(svm_cm) lr_per_prec,lr_per_recall =
get_prec_recall_per_emotion(lr_cm) rfc_per_prec,rfc_per_recall =
get_prec_recall_per_emotion(rfc_cm) mnb_per_prec,mnb_per_recall =
get_prec_recall_per_emotion(mnb_cm) dt_per_prec,dt_per_recall =
get_prec_recall_per_emotion(dt_cm)

per_precision_list = pd.DataFrame({'Support vector Machine' : svm_per_prec,
                                'Logistic Regression' : lr_per_prec,
                                'Random Forest' : rfc_per_prec,
                                'Naive Bayes' : mnb_per_prec,
                                'Decision Tree' : dt_per_prec},
                                index=svm.classes_)

```

```

[65]:
Support vector Machine Logistic Regression Random Forest \
anger          90.152          90.704          77.681
fear           88.402          87.404          71.502
joy            89.112          82.476          79.898
love           84.296          86.236          70.378
sadness        91.686          88.409          77.412
surprise       86.875          93.333          71.053

Naive Bayes Decision Tree
anger          94.649          87.376
fear           87.963          83.091
joy            63.378          91.411
love          100.000          76.008
sadness        67.402          88.229
surprise       100.000          72.566

```

6.2 5.2 - Recall of all models

```

[67]: per_recall_list = pd.DataFrame({'Support vector Machine' : svm_per_recall,
                                'Logistic Regression' : lr_per_recall,
                                'Random Forest' : rfc_per_recall,
                                'Naive Bayes' : mnb_per_recall,
                                'Decision Tree' : dt_per_recall},

```

```

index=svm.classes_)
per_recall_list

```

```

[67]:      Support vector Machine Logistic Regression Random Forest \
anger      86.232      77.778      75.242
fear      84.345      76.788      75.169
joy      94.453      95.344      77.563
love      75.413      63.430      69.215
sadness    94.519      94.286      80.933
surprise   65.258      52.582      63.380

```

```

      Naive Bayes Decision Tree
anger      34.179      85.266
fear      25.641      84.885
joy      97.028      86.974
love       7.025      81.818
sadness    92.595      90.904
surprise    1.878      76.995

```

6.3 5.3 Accuracy Visualization for all models

```

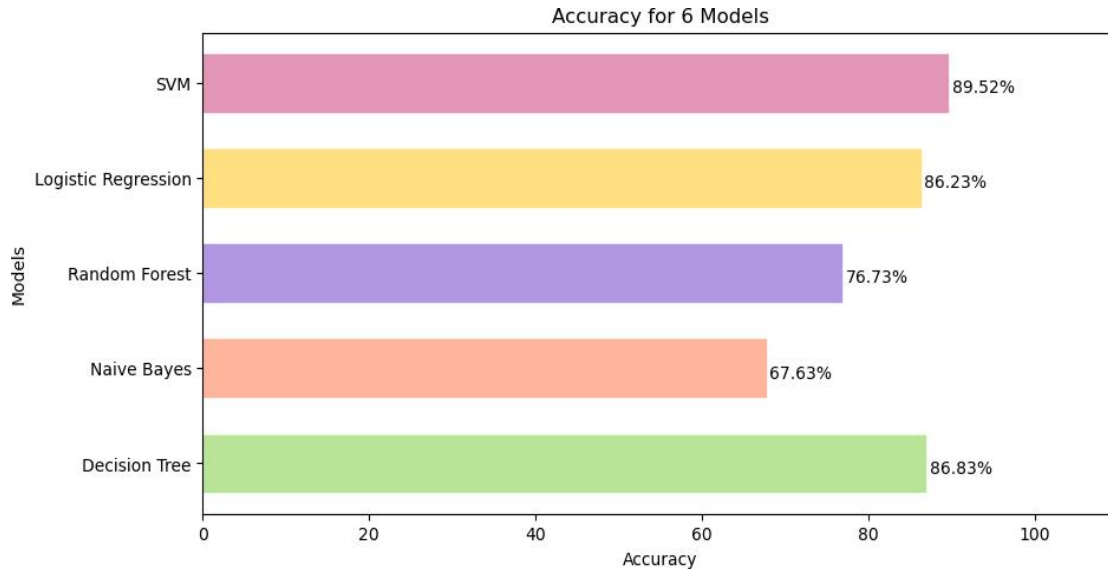
[69]: algorithms = ['SVM','Logistic Regression','Random Forest','Naive_
      #Bayes','Decision Tree']
algorithm_acc = [svm_acc*100, lr_acc*100, rfc_acc*100, mnbc_acc*100, dt_acc*100]

algorithms = ['Decision Tree','Naive Bayes','Random Forest','Logistic_
      #Regression','SVM']
algorithm_acc = [dt_acc*100, mnbc_acc*100, rfc_acc*100, lr_acc*100, svm_acc*100]
# Figure Size
fig = plt.figure(figsize =(10, 5))
bar_list = plt.barh(algorithms, algorithm_acc,height = 0.6,color='black')
bar_list[4].set_color('#E396B6')
bar_list[3].set_color('#FFDF80')
bar_list[2].set_color('#B096E3')
bar_list[1].set_color('#FFB49C')
bar_list[0].set_color('#B9E396')
plt.xlabel('Accuracy',color='black')
plt.ylabel('Models',color='black')
plt.title('Accuracy for 6 Models')
plt.xlim(0,110)

for i, v in enumerate(algorithm_acc):
    plt.text(v + 0.5, i - 0.1, str(round(v,2))+ '%', color='black',fontsize=10)

plt.show()

```



7 6. Prediction

```
[71]: while True:
    input_str = input("What's in your mind: ")
    if input_str == 'nothing':
        break

    processed_text = tfidf_transformer_xtest.fit_transform(countVectorizer1.
    transform([input_str]))
    print('Emotion using SVM: ',end = '')
    print(svm.predict(processed_text))
    print('Emotion using Logistic Regression: ',end = '')
    print(logisticRegr.predict(processed_text))
    print('Emotion using Decision Tree: ',end = '')
    print(dt.predict(processed_text))
    print('Emotion using Naive Bayes: ',end =
    '') print(mnb.predict(processed_text))
    print('Emotion using Random Forest: ',end =
    '') print(rfc.predict(processed_text))
```

What's in your mind: I won the trophy today.

Emotion using SVM: ['joy']
 Emotion using Logistic Regression: ['joy']
 Emotion using Decision Tree: ['sadness']
 Emotion using Naive Bayes: ['joy']
 Emotion using Random Forest: ['joy']

